

Université Hassan II de Casablanca
Faculté des Sciences Ben M'Sick
Département Mathématiques et Informatique

Documentation Technique

SGRH Intelligent

Système de Gestion des Ressources Humaines
avec Détection d'Anomalies par IA et Analyse OCR

PFE – Licence 3^{ème} Année Développement Informatique

Année Universitaire 2025 – 2026

Encadré par : Pr. Ichrak BENAMRI

Réalisé par : MANSOUR Aymane
FOUKOU Issa
BADIA Salahedine

Table des matières

1	Vue d'ensemble du projet	2
1.1	Présentation générale	2
1.2	Architecture globale	2
1.3	Arborescence complète du projet	3
2	Base de données (database/)	5
2.1	schema.sql — Schéma de la base de données	5
2.1.1	Tables créées	5
2.1.2	Index de performance	5
2.1.3	Conformité loi 09-08	6
2.2	seed.sql — Données de test	6
3	Frontend (frontend/)	7
3.1	Fichiers de configuration	7
3.1.1	package.json	7
3.1.2	vite.config.js	7
3.1.3	tailwind.config.js	7
3.1.4	postcss.config.js	8
3.1.5	index.html	8
3.2	Fichiers source (src/)	8
3.2.1	main.jsx — Point d'entrée React	8
3.2.2	App.jsx — Routeur principal	8
3.2.3	index.css — Styles TailwindCSS	9
3.3	Contexte (context/)	9
3.3.1	AuthContext.jsx — Gestion de l'authentification	9
3.4	Services (services/)	9
3.4.1	api.js — Client HTTP Axios	9
3.5	Composants (components/)	10
3.5.1	Layout.jsx — Mise en page	10
3.5.2	Sidebar.jsx — Menu latéral	10
3.5.3	Header.jsx — Barre supérieure	10
3.6	Pages (pages/)	10
3.6.1	LoginPage.jsx — Page de connexion	10
3.6.2	DashboardPage.jsx — Tableau de bord	10
3.6.3	EmployesPage.jsx — Gestion des employés	11
3.6.4	CongesPage.jsx — Gestion des congés	11
3.6.5	PresencesPage.jsx — Suivi des présences	11
3.6.6	PaiePage.jsx — Gestion de la paie	11

3.6.7	DocumentsPage.jsx — Documents & OCR	12
3.6.8	IAPage.jsx — Analyse IA	12
3.6.9	UsersPage.jsx — Gestion des utilisateurs (Admin)	12
4	Backend (backend/)	13
4.1	Configuration	13
4.1.1	pom.xml — Dépendances Maven	13
4.1.2	application.yml — Configuration Spring	13
4.1.3	SgrhApplication.java — Classe principale	14
4.2	Entités JPA (entity/)	14
4.2.1	Utilisateur.java	14
4.2.2	Employe.java	14
4.2.3	Conge.java	14
4.2.4	Presence.java	14
4.2.5	FichePaie.java	14
4.2.6	Document.java	15
4.2.7	Anomalie.java	15
4.2.8	Enums	15
4.3	Repositories (repository/)	15
4.4	Sécurité (security/)	15
4.4.1	JwtService.java — Service JWT	15
4.4.2	JwtAuthFilter.java — Filtre JWT	16
4.4.3	SecurityConfig.java — Configuration de sécurité	16
4.5	Contrôleurs (controller/)	16
4.5.1	AuthController.java	16
4.5.2	EmployeController.java	16
4.5.3	CongeController.java	17
4.5.4	PresenceController.java	17
4.5.5	DocumentController.java	17
4.5.6	IAController.java	17
4.5.7	DashboardController.java	17
4.6	DTOs (dto/)	17
5	Microservice IA (microservice-ia/)	18
5.1	requirements.txt	18
5.2	main.py — Serveur FastAPI	18
5.2.1	Configuration	18
5.2.2	Modèles Pydantic	18
5.2.3	Endpoint POST /detect — Isolation Forest	19
5.2.4	Endpoint POST /cluster — K-Means	19
5.2.5	Endpoint GET /rapport	19
5.2.6	Endpoint GET /generate-dataset	19
6	Fonctionnement de l'application	20
6.1	Flux d'authentification (JWT)	20
6.2	Flux de gestion des congés	20
6.3	Flux de détection d'anomalies IA	20
6.4	Flux d'analyse OCR de documents	21
6.5	Mode démonstration	21

7	Guide de démarrage	22
7.1	Prérequis	22
7.2	Étapes de lancement	22
7.2.1	Étape 1 : Base de données	22
7.2.2	Étape 2 : Backend Spring Boot	22
7.2.3	Étape 3 : Microservice IA	22
7.2.4	Étape 4 : Frontend React	23
7.3	Comptes de test	23

Chapitre 1

Vue d'ensemble du projet

1.1 Présentation générale

Le **SGRH Intelligent** est une application web complète de gestion des ressources humaines. Elle se distingue des SGRH classiques par l'intégration de deux modules innovants :

IA Détection automatique d'anomalies comportementales (retards, absences suspectes) via les algorithmes **Isolation Forest** et **K-Means**.

OCR Extraction automatique de texte depuis des documents administratifs numérisés (certificats médicaux, contrats) via le moteur **Tesseract**.

1.2 Architecture globale

L'application suit une **architecture trois tiers enrichie d'un microservice IA** :

Couche	Technologie	Port	Rôle
Présentation	React JS + Vite	3000	Interface utilisateur, tableau de bord, formulaires
Backend	Spring Boot / Java 21	8080	API REST, logique métier, sécurité JWT/RBAC
Microservice IA	FastAPI / Python	8000	Isolation Forest, K-Means, génération de rapports
OCR	Tesseract 5.x	–	Extraction de texte (PDF, images)
Données	MySQL 8	3306	Base de données relationnelle

Flux de communication

1. Le **Frontend** envoie des requêtes HTTP REST au **Backend** avec un token JWT dans le header **Authorization**.
2. Le **Backend** traite la logique métier et accède à **MySQL** via JPA/Hibernate.
3. Pour l'analyse IA, le Backend appelle le **Microservice FastAPI** via HTTP POST.
4. Pour l'OCR, le Backend invoque **Tesseract** localement sur les fichiers uploadés.
5. Les résultats sont retournés au Frontend pour affichage dans le tableau de bord.

1.3 Arborescence complète du projet

Structure des dossiers

```

1 PFE_Slah/
2 |-- frontend/                # Interface utilisateur React
3 |   |-- index.html           # Point d'entree HTML
4 |   |-- package.json         # Dependances NPM
5 |   |-- vite.config.js       # Configuration Vite
6 |   |-- tailwind.config.js   # Configuration TailwindCSS
7 |   |-- postcss.config.js    # Configuration PostCSS
8 |   +-- src/
9 |       |-- main.jsx         # Demarrage de React
10 |       |-- App.jsx          # Routeur principal
11 |       |-- index.css        # Styles TailwindCSS
12 |       |-- context/
13 |           +-- AuthContext.jsx # Gestion authentication
14 |       |-- services/
15 |           +-- api.js        # Client Axios + intercepteurs
16 |       |-- components/
17 |           |-- Layout.jsx    # Mise en page generale
18 |           |-- Sidebar.jsx  # Menu lateral
19 |           +-- Header.jsx    # Barre superieure
20 |       +-- pages/
21 |           |-- LoginPage.jsx  # Page de connexion
22 |           |-- DashboardPage.jsx # Tableau de bord
23 |           |-- EmployesPage.jsx # Gestion des employes
24 |           |-- CongesPage.jsx # Gestion des conges
25 |           |-- PresencesPage.jsx # Suivi des presences
26 |           |-- PaiePage.jsx  # Fiches de paie
27 |           |-- DocumentsPage.jsx # Documents & OCR
28 |           |-- IAPage.jsx    # Analyse IA
29 |           +-- UsersPage.jsx # Gestion utilisateurs
30 |-- backend/                 # API REST Spring Boot
31 |   |-- pom.xml              # Dependances Maven
32 |   +-- src/main/
33 |       |-- resources/

```

```
34 |         |    +-- application.yml    # Configuration Spring
35 |         +-- java/ma/fsb/sgrh/
36 |         |-- SgrhApplication.java # Classe principale
37 |         |-- entity/              # Entites JPA (7 classes)
38 |         |-- repository/          # Repositories Spring Data
39 |         |-- controller/          # Controleurs REST (6)
40 |         |-- service/              # Services metier
41 |         |-- security/            # JWT + Spring Security
42 |         +-- dto/                  # Objets de transfert
43 |-- microservice-ia/                # Module IA Python
44 |   |-- main.py                     # Serveur FastAPI
45 |   +-- requirements.txt             # Dependances pip
46 |-- database/                       # Scripts SQL
47 |   |-- schema.sql                  # Schema de la BDD
48 |   +-- seed.sql                    # Donnees de test
49 +-- README.md                       # Guide de demarrage
```

Chapitre 2

Base de données (database/)

2.1 `schema.sql` — Schéma de la base de données

Ce fichier crée la base `sgrh_intelligent` et définit les 7 tables principales, conformément au diagramme de classes UML et à la méthode Merise (MCD/MLD).

2.1.1 Tables créées

Table	Description
<code>utilisateur</code>	Comptes utilisateurs avec email, mot de passe hashé (BCrypt), rôle (ADMIN, RH, EMPLOYE) et statut actif/inactif.
<code>employe</code>	Informations personnelles et professionnelles : matricule, nom, prénom, poste, département, salaire, solde de congé. Liée à <code>utilisateur</code> via clé étrangère (relation 1 :1).
<code>conge</code>	Demandes de congé avec dates, type (ANNUEL, MALADIE, SANS_SOLDE), statut (EN_ATTENTE, VALIDE, REFUSE) et motif.
<code>presence</code>	Pointages quotidiens : date, heure d'arrivée, heure de départ, indicateur de retard. Contrainte d'unicité sur (employé, date).
<code>fiche_paie</code>	Fiches de paie mensuelles avec salaire brut et net. Contrainte d'unicité sur (employé, mois, année).
<code>document</code>	Documents uploadés : nom de fichier, type, chemin physique, texte extrait par OCR, statut de traitement.
<code>anomalie</code>	Résultats de l'analyse IA : score d'isolation, type d'anomalie, niveau de risque (FAIBLE, MOYEN, ELEVE), label de cluster.

2.1.2 Index de performance

Quatre index sont créés pour accélérer les requêtes fréquentes :

- `idx_conge_employe_statut` : recherche rapide des congés par employé et statut.
- `idx_presence_date` : filtrage des présences par date.
- `idx_anomalie_risque` : tri des anomalies par niveau de risque et date.
- `idx_employe_departement` : filtrage par département.

2.1.3 Conformité loi 09-08

La base est configurée en `utf8mb4` pour supporter le français et l'arabe. Les mots de passe sont stockés en hash BCrypt (jamais en clair). Les identifiants personnels peuvent être anonymisés avant tout traitement IA.

2.2 `seed.sql` — Données de test

Ce fichier insère des données réalistes pour tester l'application :

- **7 utilisateurs** : 1 admin, 1 RH, 5 employés (mot de passe BCrypt pour `password123`).
- **6 employés** avec des postes variés (développeur, designer, comptable, marketing, technicien).
- **5 congés** couvrant les trois statuts (validé, en attente, refusé).
- **12 présences** sur une semaine type, incluant des retards.
- **5 fiches de paie** pour mai 2025.

Chapitre 3

Frontend (frontend/)

3.1 Fichiers de configuration

3.1.1 `package.json`

Fichier de configuration NPM qui déclare :

- Le nom du projet (`sgrh-intelligent-frontend`) et sa version.
- Les **dépendances de production** : `react`, `react-dom`, `react-router-dom` (rou-tage SPA), `axios` (appels HTTP), `recharts` (graphiques), `lucide-react` (icônes), `date-fns` (dates), `react-hot-toast` (notifications).
- Les **dépendances de développement** : `vite` (bundler ultra-rapide), `tailwindcss` (framework CSS utilitaire), `autoprefixer`, `postcss`.
- Les **scripts** : `dev` (serveur de développement), `build` (production), `preview`.

3.1.2 `vite.config.js`

Configuration du bundler Vite :

- Active le plugin `@vitejs/plugin-react` pour le support JSX.
- Définit un alias `@` pointant vers `./src` pour des imports propres.
- Configure un **proxy** : toutes les requêtes `/api` sont redirigées vers `http://localhost:8080` (backend Spring Boot), évitant les problèmes CORS en développement.
- Le serveur de développement écoute sur le **port 3000**.

3.1.3 `tailwind.config.js`

Configuration de TailwindCSS :

- `content` : scanne tous les fichiers `.jsx/.tsx` dans `src/` pour générer uniquement les classes CSS utilisées (purge automatique).
- `colors.primary` : palette de bleus personnalisée (50 à 900) pour l'identité visuelle.
- `colors.sidebar` : couleurs sombres (`#0f172a`) pour le menu latéral.
- `fontFamily` : police Inter pour une typographie moderne.

3.1.4 postcss.config.js

Active les plugins PostCSS nécessaires : **tailwindcss** (compilation des classes utilitaires) et **autoprefixer** (ajout automatique des préfixes navigateurs).

3.1.5 index.html

Point d'entrée HTML de l'application :

- Charge la police **Inter** depuis Google Fonts.
- Contient un `<div id="root">` où React monte l'application.
- Référence `/src/main.jsx` comme module d'entrée JavaScript.

3.2 Fichiers source (src/)

3.2.1 main.jsx — Point d'entrée React

Ce fichier initialise l'application React :

1. Crée la racine React avec **ReactDOM.createRoot**.
2. Enveloppe l'application dans **BrowserRouter** (routage côté client).
3. Enveloppe dans **AuthProvider** (contexte d'authentification global).
4. Ajoute le composant **Toaster** pour les notifications toast.
5. Active **StrictMode** pour détecter les problèmes potentiels.

3.2.2 App.jsx — Routeur principal

Définit toutes les routes de l'application avec **React Router v6** :

Route	Composant	Accès
/login	LoginPage	Public (redirige si déjà connecté)
/dashboard	DashboardPage	Tous les rôles authentifiés
/employes	EmployesPage	ADMIN, RH uniquement
/conges	CongesPage	Tous les rôles authentifiés
/presences	PresencesPage	Tous les rôles authentifiés
/paie	PaiePage	ADMIN, RH uniquement
/documents	DocumentsPage	Tous les rôles authentifiés
/ia	IAPage	ADMIN, RH uniquement
/utilisateurs	UsersPage	ADMIN uniquement

Le composant **ProtectedRoute** vérifie :

1. Si l'utilisateur est authentifié (sinon, redirection vers `/login`).
2. Si l'utilisateur a le rôle requis (sinon, redirection vers `/dashboard`).

3.2.3 index.css — Styles TailwindCSS

Fichier CSS principal qui :

- Importe les trois couches Tailwind : `@tailwind base, components, utilities`.
- Définit des **composants réutilisables** dans `@layer components` :
 - `.btn-primary, .btn-secondary, .btn-danger` : boutons stylisés.
 - `.card` : conteneur blanc avec ombre et coins arrondis.
 - `.input-field` : champs de formulaire avec focus ring bleu.
 - `.badge-success, .badge-warning, .badge-danger, .badge-info` : étiquettes colorées.

3.3 Contexte (context/)

3.3.1 AuthContext.jsx — Gestion de l'authentification

Ce fichier implémente le **Context API** de React pour gérer l'état d'authentification globalement :

- **user** : objet contenant `id, email, role` de l'utilisateur connecté.
- **login(userData, token)** : sauvegarde l'utilisateur et le token JWT dans `localStorage`.
- **logout()** : supprime les données de `localStorage` et remet `user` à `null`.
- **getToken()** : retourne le token JWT stocké.
- **loading** : état booléen qui évite un flash de la page login pendant la vérification initiale.

Au montage du composant (`useEffect`), le contexte vérifie si un utilisateur et un token sont déjà stockés dans `localStorage` pour maintenir la session.

3.4 Services (services/)

3.4.1 api.js — Client HTTP Axios

Ce fichier centralise **tous les appels API** de l'application :

- Crée une instance Axios avec l'URL de base `http://localhost:8080/api`.
- **Intercepteur de requête** : ajoute automatiquement le header `Authorization: Bearer <token>` à chaque requête si un token JWT existe.
- **Intercepteur de réponse** : si le serveur retourne un code 401 (non autorisé), déconnecte l'utilisateur et redirige vers `/login`.
- Exporte 7 **services** organisés par module :
 - `authService` : `login(), register(), me()`.
 - `employeService` : CRUD complet (`getAll, getById, create, update, delete`).
 - `congeService` : `getAll, getByEmploye, create, valider, refuser`.
 - `presenceService` : `getAll, getByEmploye, pointer`.
 - `paieService` : `getAll, getByEmploye, generer`.
 - `documentService` : `getAll, upload (multipart/form-data), getById`.
 - `iaService` : `detecterAnomalies, segmenter, getRapport, getAnomalies`.

3.5 Composants (components/)

3.5.1 Layout.jsx — Mise en page

Composant de mise en page principal qui structure l'interface en deux zones :

- **Sidebar** (à gauche, largeur fixe 256px) : menu de navigation.
- **Zone principale** (à droite, flexible) : contient le **Header** en haut et le contenu de la page (`<Outlet />` de React Router) en dessous.

Utilise `flexbox` avec `h-screen overflow-hidden` pour occuper tout l'écran.

3.5.2 Sidebar.jsx — Menu latéral

Menu de navigation vertical avec fond sombre (`#0f172a`) :

- **Logo** : icône Brain avec le texte « SGRH Intelligent ».
- **Navigation** : liste de liens filtrés selon le rôle de l'utilisateur. Chaque lien a une icône Lucide et un label. Le lien actif est surligné en bleu (`NavLink` avec `isActive`).
- **Pied** : avatar avec l'initiale de l'email, rôle affiché, et bouton de déconnexion.
- **Filtrage par rôle** : chaque élément de navigation déclare les rôles autorisés. Seuls les éléments correspondant au rôle de l'utilisateur s'affichent.

3.5.3 Header.jsx — Barre supérieure

Barre horizontale contenant :

- Un champ de **recherche** avec icône loupe.
- Une **cloche de notifications** avec indicateur rouge.
- L'**avatar** de l'utilisateur avec son nom et rôle traduit en français.

3.6 Pages (pages/)

3.6.1 LoginPage.jsx — Page de connexion

Page divisée en deux panneaux :

- **Panneau gauche** (visible sur écrans larges) : branding avec gradient bleu, logo, description du projet, et trois cartes « IA / OCR / RH ».
- **Panneau droit** : formulaire de connexion avec email et mot de passe, icônes, bouton de visibilité du mot de passe, et indicateur de chargement.
- **Mode démo** : trois boutons (Admin / RH / Employé) permettent de se connecter **sans backend** avec des données fictives. Idéal pour la maintenance.

3.6.2 DashboardPage.jsx — Tableau de bord

Page d'accueil après connexion, composée de :

- **4 cartes statistiques** : Total employés, Congés en attente, Présents aujourd'hui, Anomalies IA. Chaque carte affiche une tendance (hausse/baisse).

- **Graphique à barres** (Recharts) : congés par mois, ventilés par type (annuels, maladie, sans solde).
- **Graphique en anneau** : répartition des employés par département.
- **Graphique d'aire** : présences et retards de la semaine.
- **Liste d'activité récente** : 5 dernières actions du système avec horodatage.

3.6.3 EmployesPage.jsx — Gestion des employés

Interface CRUD complète pour les employés :

- **Barre de recherche** filtrant en temps réel par nom, matricule ou département.
- **Tableau** avec colonnes : avatar/nom, matricule (police monospace), poste, département (badge bleu), salaire (formaté MAD), statut (badge vert/rouge).
- **Actions** par ligne : voir, modifier, supprimer (icônes avec hover coloré).
- **Modal d'ajout** : formulaire en deux colonnes (nom, prénom, email, poste, département, salaire).
- **Bouton export** pour télécharger la liste.

3.6.4 CongesPage.jsx — Gestion des congés

- **Filtres par statut** : boutons « Tous / En attente / Validé / Refusé ».
- **Cartes de congé** : chaque demande affiche l'employé, les dates, le nombre de jours, le type, le statut (badge coloré) et le motif.
- **Actions RH** : boutons Valider (vert) et Refuser (rouge) visibles uniquement pour les rôles RH/Admin sur les congés en attente.
- **Modal de nouvelle demande** : formulaire avec sélecteurs de dates, type de congé et motif.

3.6.5 PresencesPage.jsx — Suivi des présences

- **Sélecteur de date** pour filtrer par jour.
- **4 cartes** : Présents, Absents, Retards, Taux de présence (%).
- **Tableau des pointages** : employé, date, heure d'arrivée/départ (monospace), durée calculée, statut (« À l'heure » ou « En retard »).

3.6.6 PaiePage.jsx — Gestion de la paie

- **3 cartes** : masse salariale nette, salaire moyen, nombre de fiches générées.
- **Graphique à barres** : évolution de la masse salariale sur 6 mois (couleur violet).
- **Tableau des fiches** : employé, période, salaire brut, salaire net (en gras), bouton de téléchargement.
- **Bouton** « Générer les fiches » et « Exporter PDF ».

3.6.7 DocumentsPage.jsx — Documents & OCR

- **3 cartes statistiques** : total documents, OCR réussi, en traitement.
- **Grille de documents** (2 colonnes) : chaque carte affiche le nom du fichier, le type, l'employé, le statut (Traité/En cours/Erreur), et un aperçu du texte extrait.
- **Modal d'upload** : zone de drag-and-drop avec sélecteur de type de document.
- **Modal de détail** : affiche toutes les métadonnées et le texte extrait complet par OCR.

3.6.8 IAPage.jsx — Analyse IA

Page la plus riche visuellement :

- **Bouton « Lancer l'analyse »** avec animation de chargement (spinner).
- **4 cartes** : anomalies détectées, taux de contamination (5%), nombre de clusters (3), précision (92%).
- **Scatter plot** (nuage de points) : visualisation des clusters K-Means avec couleurs (vert=Normal, jaune=Retardataire, rouge=Absent).
- **Graphique en anneau** : distribution des profils comportementaux.
- **Tableau des anomalies** : employé, barre de score visuelle, type d'anomalie, niveau de risque (badge), cluster assigné (pastille colorée), date.

3.6.9 UsersPage.jsx — Gestion des utilisateurs (Admin)

- **3 cartes** : nombre d'administrateurs, comptes actifs, comptes désactivés.
- **Tableau** : email, rôle (badge coloré par rôle), statut, dernière connexion.
- **Modal de création** : email, mot de passe, sélection du rôle.

Chapitre 4

Backend (backend/)

4.1 Configuration

4.1.1 pom.xml — Dépendances Maven

Le fichier POM déclare les dépendances du projet Spring Boot :

Dépendance	Rôle
spring-boot-starter-web	Framework web, serveur Tomcat intégré, API REST
spring-boot-starter-data-jpa	ORM Hibernate, repositories auto-générés
spring-boot-starter-security	Spring Security, filtres d'authentification
spring-boot-starter-validation	Validation des DTOs avec annotations Jakarta
mysql-connector-j	Driver JDBC pour MySQL
jjwt-api/impl/jackson	Bibliothèque JWT pour génération et validation des tokens
lombok	Génération automatique de getters, setters, constructeurs
spring-boot-starter-webflux	WebClient pour appeler le microservice IA (HTTP réactif)

4.1.2 application.yml — Configuration Spring

Fichier YAML qui configure :

- **DataSource** : URL MySQL (`jdbc:mysql://localhost:3306/sgrh_intelligent`), identifiants.
- **JPA/Hibernate** : mode `ddl-auto: update` (création/modification automatique des tables).
- **JWT** : clé secrète de signature (256 bits), durée d'expiration (24h = 86400000ms).
- **Microservice IA** : URL du service FastAPI (`http://localhost:8000`).
- **OCR** : chemin vers l'exécutable Tesseract et langues (`fra+ara`).
- **Upload** : taille maximale des fichiers (10 Mo).

4.1.3 SgrhApplication.java — Classe principale

Point d'entrée de l'application Spring Boot. La méthode `main()` appelle `SpringApplication.run()` qui :

1. Scanne tous les packages sous `ma.fsb.sgrh`.
2. Configure automatiquement les beans (auto-configuration).
3. Démarre le serveur Tomcat intégré sur le port 8080.

4.2 Entités JPA (entity/)

Les entités sont des classes Java annotées `@Entity` qui correspondent directement aux tables MySQL. Lombok (`@Data`, `@Builder`) génère automatiquement les getters, setters, constructeurs et `toString()`.

4.2.1 Utilisateur.java

- Implémente `UserDetails` (interface Spring Security) pour être directement utilisable dans l'authentification.
- Champs : `id`, `email` (unique), `motDePasse`, `role` (enum), `actif`.
- La méthode `getAuthorities()` retourne le rôle avec le préfixe `ROLE_` requis par Spring Security.
- `isAccountNonLocked()` retourne la valeur de `actif`, permettant de bloquer un compte.
- Callbacks `@PrePersist` et `@PreUpdate` pour horodater automatiquement.

4.2.2 Employe.java

- Hérite conceptuellement de `Utilisateur` via une relation `@OneToOne`.
- Contient les relations `@OneToMany` vers `Conge`, `Presence`, `FichePaie`, `Document`, `Anomalie` (cascade ALL).
- Champs métier : `matricule`, `nom`, `prenom`, `poste`, `departement`, `salaire`, `soldeConge`, `dateEmbauche`.

4.2.3 Conge.java

Demande de congé avec `dateDebut`, `dateFin`, `nbJours`, `typeConge` (enum), `statut` (enum, défaut `EN_ATTENTE`), et `motif`.

4.2.4 Presence.java

Pointage quotidien avec `datePresence`, `heureArrivee` (`LocalTime`), `heureDepart`, et `estEnRetard` (booléen).

4.2.5 FichePaie.java

Fiche de paie mensuelle avec `mois`, `annee`, `salaireBase` et `salaireNet`.

4.2.6 Document.java

Document uploadé avec `nomFichier`, `typeDocument`, `cheminFichier`, `texteExtrait` (résultat OCR), et `statut` (défaut `EN_COURS`).

4.2.7 Anomalie.java

Résultat d'analyse IA avec `scoreIsolation`, `typeAnomalie`, `niveauRisque`, `clusterLabel`, `dateDetection`, et `descriptionIa`.

4.2.8 Enums

Trois énumérations Java :

- `Role` : `ADMIN`, `RH`, `EMPLOYEE`.
- `StatutConge` : `EN_ATTENTE`, `VALIDE`, `REFUSE`.
- `TypeConge` : `ANNUEL`, `MALADIE`, `SANS_SOLDE`.

4.3 Repositories (repository/)

Les repositories sont des interfaces qui étendent `JpaRepository<Entity, Long>`. Spring Data JPA génère automatiquement les implémentations SQL à partir des noms de méthodes.

Repository	Méthodes personnalisées
UtilisateurRepository	<code>findByEmail()</code> , <code>existsByEmail()</code>
EmployeRepository	<code>findByMatricule()</code> , <code>findByUtilisateurId()</code> , <code>findByDepartement()</code>
CongeRepository	<code>findByEmployeId()</code> , <code>findByStatut()</code> , <code>findByEmployeIdAndStatut()</code>
PresenceRepository	<code>findByDatePresence()</code> , <code>findByEmployeIdAndDatePresenceBetween()</code> , <code>countByDatePresenceAndEstEnRetardTrue()</code>
FichePaieRepository	<code>findByEmployeId()</code> , <code>findByMoisAndAnnee()</code>
DocumentRepository	<code>findByEmployeId()</code> , <code>findByStatut()</code>
AnomalieRepository	<code>findByEmployeId()</code> , <code>findByNiveauRisque()</code> , <code>findTop10ByOrderByDateDetectionDesc()</code>

4.4 Sécurité (security/)

4.4.1 JwtService.java — Service JWT

Gère le cycle de vie des tokens JWT :

- `generateToken()` : crée un token signé HS256 contenant l'email comme `subject`, le rôle dans les claims, et une expiration de 24h.
- `extractUsername()` : décode le token et extrait l'email.
- `isTokenValid()` : vérifie que le token n'est pas expiré et que le username correspond.
- La clé de signature est décodée depuis Base64 et utilise HMAC-SHA256.

4.4.2 JwtAuthFilter.java — Filtre JWT

Filtre Spring qui intercepte **chaque requête HTTP** :

1. Extrait le header **Authorization**.
2. Si le header commence par **Bearer** , extrait le token.
3. Décode l'email depuis le token via **JwtService**.
4. Charge l'utilisateur depuis la base de données.
5. Vérifie la validité du token.
6. Si valide, crée un objet **Authentication** et le place dans le **SecurityContext**.

4.4.3 SecurityConfig.java — Configuration de sécurité

Classe de configuration Spring Security :

- **CORS** : autorise les requêtes depuis **http://localhost:3000** (frontend React).
- **CSRF** : désactivé (API stateless avec JWT).
- **Règles d'autorisation** :
 - **/api/auth/**** : accès public (login, register).
 - **/api/employees/****, **/api/fiches-paie/****, **/api/ia/**** : rôles **ADMIN** ou **RH**.
 - **/api/utilisateurs/**** : **ADMIN** uniquement.
 - Toutes les autres routes : authentifié.
- **Session** : mode **STATELESS** (pas de session HTTP, authentification par token uniquement).
- **Password encoder** : **BCryptPasswordEncoder** pour le hashage des mots de passe.

4.5 Contrôleurs (controller/)

4.5.1 AuthController.java

- **POST /api/auth/login** : authentifie l'utilisateur, génère un token JWT, retourne le token et les infos utilisateur.
- **POST /api/auth/register** : crée un nouveau compte avec le rôle **EMPLOYEE** par défaut.

4.5.2 EmployeController.java

CRUD complet sur les employés :

- **GET /api/employees** : liste tous les employés.
- **GET /api/employees/{id}** : détail d'un employé.
- **POST /api/employees** : création (ADMIN/RH).
- **PUT /api/employees/{id}** : modification (ADMIN/RH).
- **DELETE /api/employees/{id}** : suppression (ADMIN uniquement).

4.5.3 CongeController.java

- **GET** /api/conges et /api/conges/employe/{id} : liste des congés.
- **GET** /api/conges/en-attente : congés en attente de validation.
- **POST** /api/conges : soumission d'une demande (statut EN_ATTENTE).
- **PUT** /api/conges/{id}/valider : valide le congé et **décrémente le solde** de l'employé.
- **PUT** /api/conges/{id}/refuser : refuse le congé.

4.5.4 PresenceController.java

- **GET** par employé ou par date.
- **POST** /api/presences : enregistrement d'un pointage. Détecte automatiquement si l'employé est en retard (arrivée après 08h30).

4.5.5 DocumentController.java

- **GET** /api/documents et /api/documents/{id}.
- **POST** /api/documents/upload : reçoit un fichier multipart/form-data, le sauvegarde dans le dossier uploads/, crée l'entrée en base avec statut EN_COURS. L'appel à Tesseract OCR est prévu en asynchrone.

4.5.6 IAController.java

Pont entre le backend et le microservice IA Python :

- **POST** /api/ia/detect : appelle **POST** http://localhost:8000/detect via WebClient.
- **POST** /api/ia/cluster : appelle **POST** http://localhost:8000/cluster.
- **GET** /api/ia/rapport : récupère le rapport consolidé depuis FastAPI.
- **GET** /api/ia/anomalies : retourne les 10 dernières anomalies depuis MySQL.

4.5.7 DashboardController.java

- **GET** /api/dashboard/stats : retourne les 4 indicateurs clés (total employés, congés en attente, présents aujourd'hui, anomalies).

4.6 DTOs (dto/)

- **AuthRequest** : objet de requête login/register avec validation (@Email, @NotBlank).
- **AuthResponse** : objet de réponse contenant le token JWT et un sous-objet UserDto (id, email, role).

Chapitre 5

Microservice IA (`microservice-ia/`)

5.1 `requirements.txt`

Dépendances Python du microservice :

Package	Version	Rôle
<code>fastapi</code>	0.104.1	Framework web asynchrone haute performance
<code>uvicorn</code>	0.24.0	Serveur ASGI pour FastAPI
<code>scikit-learn</code>	1.3.2	Algorithmes ML (Isolation Forest, K-Means, StandardScaler)
<code>pandas</code>	2.1.4	Manipulation de données tabulaires (DataFrames)
<code>numpy</code>	1.26.2	Calcul numérique
<code>matplotlib</code>	3.8.2	Génération de graphiques pour les rapports
<code>pydantic</code>	2.5.2	Validation et sérialisation des données (modèles de requête/réponse)

5.2 `main.py` — Serveur FastAPI

5.2.1 Configuration

- CORS activé pour `localhost:3000` (React) et `localhost:8080` (Spring Boot).
- Le serveur écoute sur le **port 8000**.
- Documentation Swagger automatique disponible sur `http://localhost:8000/docs`.

5.2.2 Modèles Pydantic

Modèles de validation des données entrantes et sortantes :

- **PresenceRecord** : un enregistrement de présence avec `employe_id`, `heure_arrivee`, `heure_depart`, `duree_travail`, `est_retard`, `nb_absences_semaine`.
- **DetectRequest** : liste de `PresenceRecord` + taux de contamination (défaut 5%).
- **ClusterRequest** : liste de `PresenceRecord` + nombre de clusters (défaut 3).
- **AnomalyResult** : résultat pour un employé (score, anomalie, cluster, risque).

5.2.3 Endpoint POST /detect — Isolation Forest

Algorithme de détection d'anomalies non supervisé :

1. Reçoit un dataset JSON de présences (minimum 10 enregistrements).
2. Extrait les 5 features : heure d'arrivée, heure de départ, durée de travail, retard, absences.
3. Entraîne un modèle **IsolationForest** avec 100 estimateurs et un taux de contamination de 5%.
4. Calcule le **decision_function** (score brut) et normalise entre 0 et 1 (1 = plus anormal).
5. Applique en parallèle **K-Means** (k=3) après normalisation **StandardScaler**.
6. Attribue un niveau de risque : score > 0.7 = ÉLEVÉ, > 0.5 = MOYEN, sinon FAIBLE.
7. Retourne uniquement les anomalies détectées avec leur score, cluster et niveau de risque.

5.2.4 Endpoint POST /cluster — K-Means

Segmentation des profils comportementaux :

1. Reçoit le dataset JSON.
2. Normalise les features avec **StandardScaler** (moyenne 0, écart-type 1).
3. Applique **K-Means** avec 3 clusters.
4. Attribue un label à chaque cluster :
 - **Cluster 0 - Normal** : présence régulière, peu de retards.
 - **Cluster 1 - Retardataire** : retards fréquents, irrégularités d'horaires.
 - **Cluster 2 - Absent** : taux d'absence élevé.
5. Retourne les labels par employé, les centres des clusters et la distribution.

5.2.5 Endpoint GET /rapport

Retourne un rapport consolidé avec le résumé des analyses et des recommandations RH :

- Nombre d'employés analysés et anomalies détectées.
- Distribution des clusters.
- Recommandations d'actions (entretiens, ajustement d'horaires).

5.2.6 Endpoint GET /generate-dataset

Génère un **dataset synthétique** pour les tests :

- 100 employés × 180 jours = 18 000 enregistrements.
- 95% des employés ont un comportement normal (arrivée ~ 8h00, départ ~ 17h00).
- 5% (employés 1 à 5) ont un comportement anormal injecté (arrivée ~ 9h30, départ ~ 16h00, plus d'absences).
- Utilise `numpy.random` avec une seed fixe (42) pour la reproductibilité.

Chapitre 6

Fonctionnement de l'application

6.1 Flux d'authentification (JWT)

1. L'utilisateur saisit son email et mot de passe sur la page `LoginPage`.
2. Le frontend envoie un `POST /api/auth/login` avec les identifiants.
3. Le backend vérifie les identifiants via `AuthenticationManager` (comparaison `BCrypt`).
4. Si valide, `JwtService.generateToken()` crée un token JWT signé HS256 contenant l'email et le rôle, valide 24h.
5. Le token et les infos utilisateur sont retournés au frontend.
6. Le frontend stocke le token dans `localStorage` via `AuthContext.login()`.
7. Pour chaque requête suivante, l'intercepteur `Axios` ajoute `Authorization: Bearer <token>`.
8. Côté backend, `JwtAuthFilter` intercepte chaque requête, décode le token, et place l'authentification dans le `SecurityContext`.

6.2 Flux de gestion des congés

1. L'employé ouvre la page `CongesPage` et clique sur « Nouvelle demande ».
2. Il remplit le formulaire (dates, type, motif) et soumet.
3. Le frontend envoie un `POST /api/conges` avec le statut `EN_ATTENTE`.
4. Le congé est sauvegardé en base de données.
5. Le responsable RH voit les congés en attente (filtré par statut).
6. Il clique sur « Valider » → `PUT /api/conges/{id}/valider` :
 - Le statut passe à `VALIDE`.
 - Le `soldeConge` de l'employé est décrémenté de `nbJours`.
7. Ou il clique sur « Refuser » → le statut passe à `REFUSE`.

6.3 Flux de détection d'anomalies IA

1. Le responsable RH ouvre la page `IAPage` et clique sur « Lancer l'analyse ».
2. Le frontend envoie un `POST /api/ia/detect` au backend Spring Boot.

3. Le backend récupère les données de présence depuis MySQL.
4. Le backend transmet ces données au microservice FastAPI via WebClient (POST `http://localhost:8000/detect`).
5. FastAPI exécute l'algorithme **Isolation Forest** :
 - Entraîne le modèle sur les 5 features de présence.
 - Identifie les points aberrants (score d'anomalie normalisé).
6. En parallèle, FastAPI applique **K-Means** (k=3) pour segmenter les profils.
7. Les résultats (anomalies + clusters) sont retournés au backend.
8. Le backend sauvegarde les anomalies dans la table **anomalie** de MySQL.
9. Les résultats sont envoyés au frontend qui les affiche :
 - **Scatter plot** des clusters.
 - **Tableau** des anomalies avec barres de score visuelles.
 - **Anneau** de distribution des profils.

6.4 Flux d'analyse OCR de documents

1. L'utilisateur ouvre **DocumentsPage** et clique sur « Uploader un document ».
2. Il glisse un fichier (PDF, PNG ou JPG) dans la zone de drop ou le sélectionne.
3. Le frontend envoie un POST `/api/documents/upload` en `multipart/form-data`.
4. Le backend :
 - (a) Sauvegarde le fichier physiquement dans le dossier **uploads/**.
 - (b) Crée une entrée en base avec le statut **EN_COURS**.
 - (c) Invoque Tesseract OCR sur le fichier (en asynchrone).
5. Tesseract analyse l'image/PDF et extrait le texte (français et arabe supportés).
6. Le texte extrait est sauvegardé dans le champ **texte_extrait** de la table **document**.
7. Le statut passe à **TRAITE** (ou **ERREUR** si échec).
8. L'utilisateur peut consulter le texte extrait dans le modal de détail.

6.5 Mode démonstration

L'application intègre un **mode démo** qui permet de la présenter **sans aucun backend** :

- Sur la page de login, trois boutons (Admin / RH / Employé) créent un utilisateur fictif avec un faux token.
- Toutes les pages contiennent des **données mock** (tableaux et graphiques pré-remplis).
- Les actions (ajout, validation, lancement IA) affichent des **notifications toast** de succès.
- Ce mode est idéal pour la **soutenance** si le backend ou la base de données ne sont pas disponibles.

Chapitre 7

Guide de démarrage

7.1 Prérequis

Outil	Version	Installation
Node.js	$\geq 18.x$	https://nodejs.org
Java JDK	21	https://adoptium.net
Python	3.11+	https://python.org
MySQL	8.x	https://dev.mysql.com/downloads/
Tesseract	5.x	https://github.com/tesseract-ocr/tesseract

7.2 Étapes de lancement

7.2.1 Étape 1 : Base de données

Initialisation MySQL

```
1 mysql -u root -p < database/schema.sql
2 mysql -u root -p < database/seed.sql
```

7.2.2 Étape 2 : Backend Spring Boot

Lancement du backend (port 8080)

```
1 cd backend
2 ./mvnw spring-boot:run
```

7.2.3 Étape 3 : Microservice IA

Lancement du microservice IA (port 8000)

```
1 cd microservice-ia
2 pip install -r requirements.txt
3 python main.py
```

La documentation Swagger est disponible sur <http://localhost:8000/docs>.

7.2.4 Étape 4 : Frontend React

Lancement du frontend (port 3000)

```
1 cd frontend
2 npm install
3 npm run dev
```

7.3 Comptes de test

Email	Mot de passe	Rôle	Accès
admin@sgrh.ma	password123	ADMIN	Accès complet à toutes les fonctionnalités
rh@sgrh.ma	password123	RH	Gestion RH, IA, paie (pas la gestion des comptes)
ahmed.benali@sgrh.ma	password123	EMPLOYE	Congés, présences, documents personnels